

CSC458H1: Computer Networks — Fall 2026

Programming Assignment 1: Building a Simple Router: ARP, Static Forwarding, and Traceroute

Due Date: Oct 16, 2026, 11:59 PM

Please check out the [assignment policy page](#) on Quercus for details on CSC458 policy on collaboration and code use. Note that the assignment is expected to be submitted in 3 different milestones:

- Part I is due **Friday, Oct 2, 11:59 PM**
- Part II is due on **Friday, Oct 9, 11:59 PM**
- The final due date is **Oct 16, 2026, 11:59 PM**

Overview

PA0 introduced the CSC458 virtual machine, Python, sockets, and a minimal HTTP client. PA1 now builds a small forwarding stack from the link layer upward. The three parts are deliberately connected:

1. **Network Interface + ARP:** given an IPv4 datagram and an IP next hop, determine the next-hop MAC address and transmit the datagram in an Ethernet frame;
2. **IP Forwarding + Static Routing + Longest Prefix Matching (LPM):** given an incoming IPv4 datagram, determine the output interface and IP next hop, decrement TTL, and call the Part I network interface; and
3. **TTL, ICMP, and Traceroute:** study what real routers do when TTL expires and use supplied packet parsers to interpret a traceroute capture.

Part III then asks what happens when a forwarded packet's TTL reaches zero. Your Part II router silently drops it; real routers generally return ICMP Time Exceeded, which is the mechanism exploited by traceroute.

Make sure to use the official CSC458 Ubuntu 22.04 virtual machine. The graded tasks need no Internet access, Mininet, or root privileges. An optional live traceroute can require raw-socket privileges and may be affected by NAT, VPNs, firewalls, or ICMP filtering.

Important. You are **not** asked to implement Ethernet, ARP, IPv4, UDP, or ICMP byte parsers. Complete implementations are supplied in `protocols.py`. The programming work in PA1 is about protocol behavior, forwarding state, and packet-level reasoning rather than header-decoding boilerplate.

Learning objectives

By the end of this assignment you should be able to:

- distinguish an end-to-end IPv4 destination from an IP next hop and from an Ethernet destination MAC;
- implement ARP resolution, learning, caching, request suppression, and expiry;
- implement a static IPv4 forwarding table and longest-prefix matching (LPM);
- distinguish directly connected routes from routes through another router;
- explain TTL processing in a router and the conditions under which a packet is dropped;

- explain ICMP Time Exceeded and Port Unreachable in the context of UDP traceroute; and
- reconstruct a traceroute from a PCAP using staff-provided packet parsers.

1 Part I – Network Interface + ARP

[100 points]

1.1 Background Review

One of the key ideas of the networking stack, which has been explained in detail in the course, is to hide details of lower layers from higher levels and make it possible to have different upper layers that rely on the same lower layer. In this assignment, we focus on the Data Link layer which is responsible for creating and sending out frames and also receiving and processing incoming frames. In particular, we will focus on Ethernet frames. Your task is to complete a simplified network interface to perform **creating and sending Ethernet frames** and **receiving Ethernet frames and extracting the frame content**.

Before we explain the tasks in this part, let us briefly review how a packet traverse the network to reach its destination. Consider the network topology illustrated in Figure 1, in which Host A and B, together with Router's Interface 1 are in the same physical network, which means that they can communicate with each other at the data link layer (*i.e.*, send frames to each other) without requiring a router. In other words, communication happens through switching, not routing. Similarly, Router's Interface 2 and the Server are in the same physical network. Assume that Host A wants to send a packet to the Server.

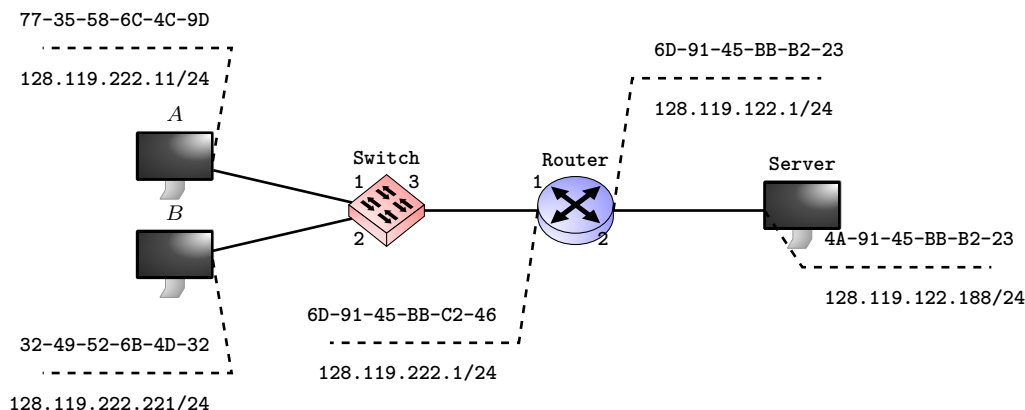


Figure 1: Host A and B and Router's Interface 1 are in the same physical network, and Router's Interface 2 and the Server are in another physical network. Host A sends one packet to the Server.

Figure 2a illustrates the networking stack for this topology. Recall that packets get additional headers wrapped around them as they move down the stack, to lower layers. To send an IP packet, we first fill in its destination IP at Layer 3. Then, we pass that packet down to Layer 2, where we have to add a MAC address to send the packet along the link. For instance, consider the packet on wire at time t . Figure 2b illustrates the L2 and L3 address fields of this packet. What MAC address should Host A use for the destination MAC address field?

1.1.1 Create and Send Ethernet Frame

The first task of the link layer that you will implement in this part is to create and send Ethernet frame. When the protocol in the upper layer (*i.e.*, IP) has a packet to send, it first must determine who the next hop is to send the packet to (*i.e.*, the next machine/router on the network that should process it). The next hop is a machine that resides in the same physical network, which means that we can directly send packets to it. The next hop is determined as part of the packet routing. For instance, Consider

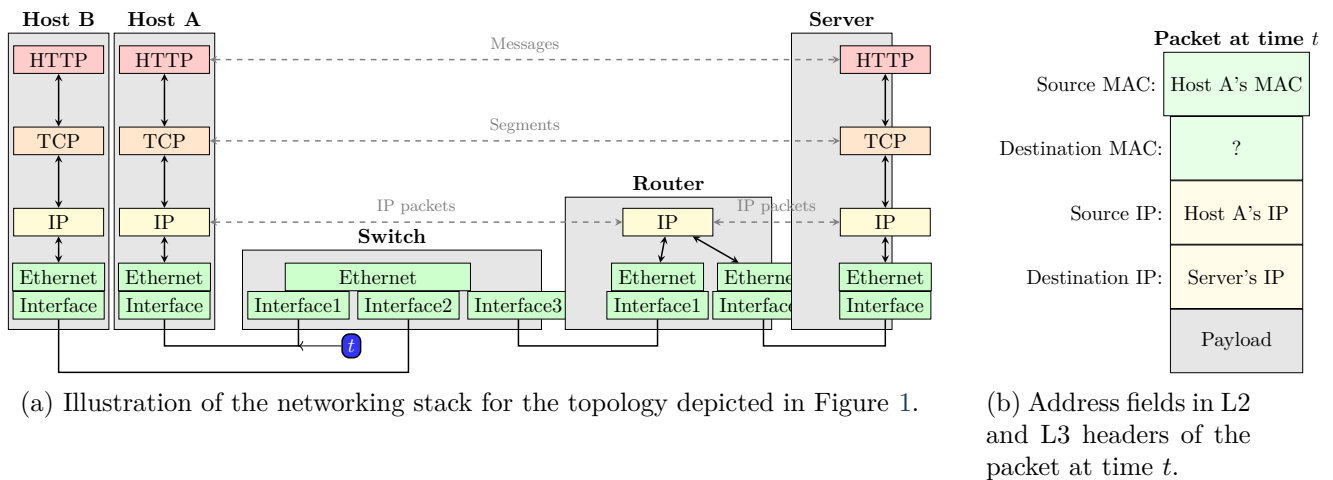


Figure 2: What MAC address should Host A use for the destination MAC address field?

the example in Figure 1. When IP in Host A wants to send a packet to the Server, the next hop will be determined as the Router's Interface 1. In this part, we assume that the next hop has been given to us. In the next part, we study how the next hop is determined.

Next, IP will invoke the link layer to send that packet to this next hop and provides the IP packet and the IP address of the next hop to the link layer. For instance, IP in Host A in Figure 1 will invoke the link layer to send the packet, which has Source IP address 128.119.222.11 and destination IP address 128.119.122.188 (*i.e.*, Server's IP address), to the next hop which has IP address 128.119.222.1 (*i.e.*, the IP address of the Router's Interface 1).

The network layer only provides the IP address of the next hop. However, the link layer requires the MAC address of the next hop to communicate with it. Thus, the first job of the link layer is to figure out the corresponding MAC address of this IP address.

Address Resolution Protocol (ARP) allows machines to translate an IP address into its corresponding MAC address. Link layer maintains a **ARP Table** which translates IP addresses to MAC addresses. This table contains the MAC addresses of different IP addresses that this machine is aware of.

If the IP address of the next hop is available in the table (*i.e.*, the machine knows the corresponding MAC address for it), it can easily send the packet out by creating an Ethernet frame with appropriate header sections (such as its own MAC address as the source and the MAC address of the next hop as destination) and include the given IP packet as the content of the frame.

If the next hop's IP address is not available in the table, the link layer will try to find the MAC address of this IP address first. To do so, ARP is used to find the MAC address of an IP address.

- To request a translation, a machine can **broadcast** a solicitation message: "I have MAC address 77-35-58-6C-4C-9D. What is the MAC address of the machine with IP 128.119.222.18?" In the example illustrated in Figure 1, this broadcast frame will be received by Host B and Router's Interface 1.
- All machines who are not this IP address ignore the message. The user who has this IP address unicasts a reply to the sender's MAC address, saying "I am 128.119.222.18, and my MAC address is 6D-91-45-BB-C2-46."¹
- When the machine receives an IP-to-MAC mapping, it adds it to the local **ARP Table**, which

¹Machines can also broadcast their own IP-to-MAC mapping to everybody, even if nobody asks.

caches these mappings for the future. The table also includes an expiry date for each entry, since IP addresses aren't permanently assigned to a computer. A different computer could get assigned the same IP address, or the same computer could change IP addresses over time. After expiry of an entry, it will be removed from the table. In this assignment, **use 30 seconds as the expiry time**.

Note that ARP runs directly on Layer 2, so all packets are sent and received over Ethernet, not IP.

If it could find the MAC address of this IP address, it sends the waiting packet out. However, if it could not find the MAC address of this IP address after the maximum wait time, it will assume that the next hop is not active anymore and drops the packet. In this assignment, **use 5 seconds as the maximum wait time**.

1.1.2 Receive Ethernet Frames and Extract Frames Content

The other task of the link layer that you must implement in this assignment is to process the incoming Ethernet packets. When it receives an Ethernet packet that is addressed to its machine, it will try to extract the content of the frame and pass it to the upper layer (*i.e.*, IP). It also has to process incoming ARP requests and answer anyone that relates to it.

1.2 Summary of The Required ARP behavior

Your interface maintains an ARP cache and per-next-hop pending state.

- If the next-hop IP is cached, encapsulate the datagram in an Ethernet frame and queue it for transmission.
- Otherwise, broadcast one ARP request and queue the datagram while resolution is pending.
- While a request for the same IP has been outstanding for less than **5 seconds**, do not send another ARP request; append additional datagrams to that pending queue.
- Learn the sender IP-to-MAC mapping from every valid ARP request or reply that reaches this interface. Cache the mapping for **30 seconds**.
- Learning a mapping releases all datagrams waiting for that IP in FIFO order.
- If an ARP request targets this interface's IP, send a unicast ARP reply.
- At 30 seconds, an ARP-cache entry expires. At 5 seconds, unresolved ARP state expires and all datagrams waiting behind it are dropped.

ARP runs directly over Ethernet, not inside IPv4.

1.3 Task I

In task 1, you should modify only `network_interface.py` and the report if needed.

`protocols.py` supplies `EthernetFrame`, `ARPMessage`, and `IPv4Packet`, including serialization/parsing. For example, the following shows the python code to create an `EthernetFrame` for ARP request.

```
request = ARPMessage.request(my_mac, my_ip, target_ip)
frame = EthernetFrame(
    dst=ETHERNET_BROADCAST,
    src=my_mac,
    ethertype=ETHERTYPE_ARP,
    payload=request.to_bytes(),
)
```

Do not reimplement these packet formats.

Implement the following methods. Read the docstrings and comments in `network_interface.py` for more instructions and details.

```
send_datagram(datagram: IPv4Packet, next_hop_ip: str) -> None
recv_frame(frame: EthernetFrame) -> IPv4Packet | None
maybe_send() -> EthernetFrame | None
tick(ms_since_last_tick: int) -> None
```

Hidden tests include multiple simultaneous unresolved next hops, FIFO release of several waiting datagrams, replacement/refresh of learned mappings, malformed input, and frames addressed to another host. Avoid hard-coding addresses or assuming that only one ARP resolution can be active.

1.4 Submission for Part I

Please submit the following by **Friday, October 2, 2026, at 11:59 PM**

- `network_interface.py`

2 Part II – IP Forwarding + Static Routing + LPM [100 points]

Part I assumes that some Layer-3 component has already selected the next-hop IP. Part II implements that component. Thus, Part II selects the *IP next hop*; Part I resolves it to a *MAC address* and produces the Ethernet frame.

2.1 Longest-prefix matching

A route is an IPv4 prefix such as `10.4.32.0/20`. A route matches destination address d if the first p bits of d equal the first p bits of the route prefix, where p is the prefix length. If several routes match, choose the one with the largest prefix length.

For example, with

Prefix	Next hop	Output interface
0.0.0.0/0	192.0.2.1	0
10.0.0.0/8	192.0.2.2	1
10.4.0.0/16	192.0.2.3	2
10.4.32.0/20	directly connected	2

destination `10.4.35.7` selects `10.4.32.0/20`, not the default, `/8`, or `/16` route.

The router/host will search through its routing table to find all matching routes for the destination IP address of the packet. If there is no such route, the packet will be dropped. If there are more than one such route, it will select the route with the longest prefix (hence the name longest prefix matching).

Therefore, you should be able to perform three tasks to route a packet in a router:

- **Installing a new route:** you should have a routing table and be able to install new routes in it.
- **Finding the best matching routes:** you should then search in the routing table and find all routes that match the destination IP address of a packet. If more than one route matches the packet, then select the one with the longest prefix.
- **Send the packet out of the router:** The final action is a direct call to the Part I method:

```
self.interfaces[interface_num].send_datagram(forwarded_packet, next_hop_ip)
```

2.2 Task II.1 – IPv4 prefix helpers

[30 points]

Implement in `prefix_tools.py`:

```
canonical_prefix(cidr: str) -> str
prefix_contains(cidr: str, ip: str) -> bool
prefix_length(cidr: str) -> int
longest_prefix_match(prefixes: list[str], destination_ip: str) -> str | None
```

You may use Python’s standard-library `ipaddress` module. Your code must handle the full range /0 through /32.

2.3 Task II.2 – Install static routes and Forward queued datagrams

[70 points]

A route stores a canonical prefix, the output interface, and an optional next-hop IP. If `next_hop` is `None`, the destination network is directly attached. Adding a route for an already-installed canonical prefix replaces the previous entry. The provided method `receive_datagram(interface_num, datagram)` queues an IPv4 datagram that arrived at one router interface.

Implement the following two methods in `router.py` to add routes to the routing table and process every queued datagram:

```
Router.add_route(route_prefix: str, prefix_length: int,
                 next_hop: str | None, interface_num: int) -> None
Router.route(self) -> None
```

Do not generate ICMP upon TTL expiry in Part II. That deliberate simplification becomes the starting point of Part III.

At least one hidden test runs your actual `Router` on top of your actual Part I `NetworkInterface`. A correct route through a remote next hop should therefore cause the selected output interface to emit an ARP request when the next-hop MAC is unknown. This is the intended stack boundary:

LPM → next-hop IP → ARP → next-hop MAC.

Most Part II public tests use a small recording interface so that routing marks do not depend entirely on a perfect Part I solution.

2.4 Submission for Part II

Please submit the following by **Friday, October 9, 2026, at 11:59 PM**

- `prefix_tools.py`
- `router.py`

3 Part III – TTL, ICMP, and Traceroute

[50 points]

3.1 From your Part II drop rule to real router behavior

Your simplified implementation drops a datagram when decrementing TTL would produce zero. A real IPv4 router generally sends an ICMP Time Exceeded message back toward the source. UDP traceroute exploits exactly this behavior by sending probes with TTL values 1, 2, 3, ...

When a probe finally reaches the destination, the selected UDP destination port normally has no listening application. The destination therefore returns ICMP Destination Unreachable / Port Unreachable. In the supplied `traceroute.py` this final 3/3 message is treated as evidence that the destination has been reached.

An ICMP error quotes the beginning of the original packet. For the supplied captures in the `pcaps` directory, a probe with TTL t uses UDP destination port (`base_port + t`). The quoted UDP port therefore lets the analysis associate an asynchronous ICMP error with the probe that caused it.

Important. All Ethernet, ARP, IPv4, UDP, and ICMP parsers are supplied. The full live traceroute control loop and ICMP-reply validator are also supplied in `traceroute.py`. You are not being graded on reimplementing packet parsers or a large traceroute program.

3.2 Task III.1 – Small PCAP-analysis task [20 points]

In `analysis.py`, read the provided docstring to implement:

```
analyze_traceroute_pcap(path, source_ip, destination_ip, base_port=33434, max_ttl=30)
```

Use `traceroute_integrated.pcap` first. Its first two records are an ARP request/reply, followed by the traceroute probes/replies. Then inspect `traceroute_messy.pcap`, which contains duplicates, malformed/unrelated traffic, a wrong ICMP code, and a missing hop. Malformed and unrelated records must be ignored rather than causing the analysis to fail.

3.3 Task III.2 – PCAP interpretation and short answers [30 points]

Complete Questions 7–13 in `report/answers_template.md`.

3.4 Optional live run

The fully supplied `traceroute.py` can perform a live UDP traceroute using raw ICMP reception. After the deterministic tasks pass, you may try:

```
sudo env PYTHONPATH=src python3 -m csc458_pa1.traceroute example.com
```

Use only destinations permitted by course staff. Live behavior is not correctness-graded because NATs, firewalls, VPNs, and remote ICMP policies vary.

4 Final Submission

For the final submission, students should submit all four Python files plus the completed report.

- `network_interface.py`;
- `prefix_tools.py`;
- `router.py`;
- `analysis.py`; and
- your completed report.

Do not submit `protocols.py`, `pcap.py`, `io.py`, `traceroute.py`, PCAPs, caches, or a virtual environment.

Before submission run:

```
python3 -m pytest -q
```